

RSCharter: A Framework for Extracting Statechart Diagram Elements from the Requirements Specification

Janvi Bhatt^{a,1}, Sourish Dasgupta^{b,*,2}, Saurabh Tiwari^{c,3} and Akhilesh Sharma^{d,4}

^aSoftware Engineering Research Lab, DA-IICT, Gandhinagar, India

^bKnowledge Discovery & Management Lab, DA-IICT, Gandhinagar, India

^cSoftware Engineering Research Lab, DA-IICT, Gandhinagar, India

^dSAC ISRO, Government of India, Ahmedabad, India

ARTICLE INFO

Keywords:

Analysis and Modeling

State Diagram

Natural Language Processing (NLP)

Requirement Templates (RTs)

ABSTRACT

Context: The automated translation of design models from the requirements written in natural language (NL) textual format is tedious. An NL is inherently ambiguous, whereas a formal language is unambiguous.

Objective: In this paper, we propose a framework, **Requirement Specification Charter** (RSCharter), for translating NL textual specifications into formal First-Order Predicate Logic (FOPL) by identifying and representing them as state diagram elements.

Method: The framework used a classical NLP rule-based baseline model for identifying the statechart diagram elements from the NL.

Results: We found that the proposed approach generates FOPL and develops a state diagram with an average precision of 89% and a recall of 87%.

Conclusion: Our results depend on the complexity levels of the evaluation datasets, the dependency parser accuracy, and the coverage of the FOPL conversion rules.

1. Background and Motivation

Manual derivation of behavioral models from natural language (NL) requirements is time-consuming, error-prone, and typically performed by experienced analysts Arora et al. (2016), Bozyigit et al. (2024). Various methods have been proposed for generating UML diagrams from NL specifications using rule-based NLP techniques, but these are manual and rely on heuristic rules. Ahmed et al. Ahmed et al. (2022) reviewed systems for automatically transforming NL into UML models, identifying limitations due to NLP capabilities and heuristic-based approaches. Arora et al. Arora et al. (2016) noted the laborious nature of manual domain model creation, while automatic extraction techniques face challenges in industrial settings. Bozyigit et al. Bozyigit et al. (2024) evaluated two text-to-model tools—ChatGPT and txt2Model—both of which showed lower performance and high variance for relation types.

In this paper, we propose a framework called **Requirement Specification Charter** (RSCharter) for automated extraction of elements of the statechart diagram (such as conditions, events, actions, and states) from the NL textual specifications. RSCharter converts an NL SRS document (RUPP or EARS compliant) into equivalent First Order Predicate Logic (FOPL) expressions before generating the statechart diagram using any conventional generator. The motivation is to develop a framework for describing system behaviours and exhibit formalization with precisely defined syntax and semantics by understanding the behavioural pattern through state variables and transitions.

2. System Overview

This section describes the RSCharter pipeline consisting of three modules: (i) *Action & Trigger Extractor* (ATE), (ii) *First-Order Predicate Logic Converter* (FLC), and (iii) *State Diagram Generator* (SDG). An SRS document is first processed by a Requirement Template Normalizer (RTN) to produce a RUPP or EARS compliant Requirement Template

*Corresponding author

✉ janvi_bhatt@daiict.ac.in (J. Bhatt); sourish_dasgupta@daiict.ac.in (S. Dasgupta); saurabh_t@daiict.ac.in (S. Tiwari); akhilesh@sac.isro.gov.in (A. Sharma)

🌐 (J. Bhatt); <https://sites.google.com/view/saurabhtiwari> (S. Tiwari)

ORCID(s):

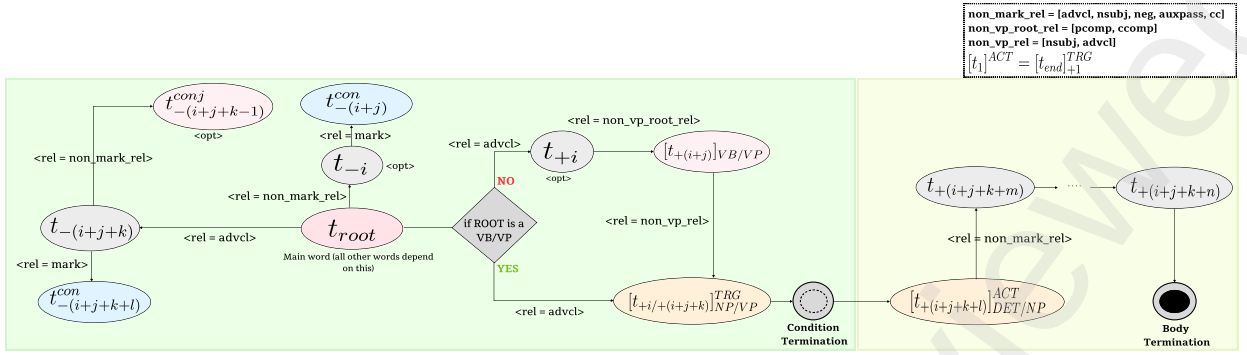


Figure 1: Action & Trigger Extractor (ATE): t : token; **subscript**: position from *root* token (-ve: left-traversal); **superscript**: {*con*: conditional token (e.g., "if", "when", "while", "in case of"), *conj*: conjunctive/disjunctive token (e.g., "and", "or"), [*TRG*]: terminating token of trigger-clause, [*ACT*]: starting token of action-clause}, **edge**: dependency relation (**rel**) as per Universal Dependency Tag Scheme (de Marneffe et al., 2014).

(RT)¹. RSCharter then uses ATE to extract Actions and Conditions (Triggers), FLC to generate FOPL expressions, and SDG to create the statechart diagram with a standard tool. Detailed functionality of each module follows.

2.1. Action & Trigger Extractor (ATE)

The ATE module uses a dependency parser-based algorithm for extracting conditional clauses (i.e., the *antecedent*) and corresponding body clauses (i.e., *consequent*). In the context of SRS, the condition clause represents the *trigger clause*, outlining preconditions for actions described in the body clause (i.e., *action clause*). Dependency parsing identifies the causal relationship between these clauses and clarifies inter-phrase associations, helping to identify compound triggers/actions involving multiple sub-clauses (see Figure 1).

Trigger Clause: There are two types of trigger clauses in a conditional SRS: (i) *existential*, where a truth value is assigned to the existence of a system state, and (ii) *comparative*, where a truth value is assigned to a comparison between states. In both cases, if the condition is true, it triggers system actions. ATE extracts trigger clauses by traversing left from the root token to find the start and right from the right token to find the end.

Action Clause: The action clause consists of what the system is supposed to perform, which amounts to the new state in which the system should transition if the trigger occurs. The action clause is extracted by traversal to the right of the token that denotes the termination of the trigger clause.

2.2. First-Order Predicate Logic Converter (FLC)

The FLC module converts the extracted action clause and associated trigger clauses (if any) into FOPL expressions. There are three types of trigger rules - one for comparative trigger clause and two for existential trigger clause (one for the case where the subject-object relation is "is-a" and the other for "non is-a" subject-object relation). We use three reserved predicates - (i) **AGENT**(•) to denote any system whose name identity is unknown (or unresolved), (ii) **STATUS**(•) to denote a property that is a state-value of an agent (e.g.: "Database x is open" implies **STATUS**(open)), and (iii) **hasStatus**(•, •) to denote the relation between the agent and its state-value (see Table 1).

2.3. State Diagram Generator (SDG)

The SDG module processes the FOPL expressions of the action and trigger clauses, converting them into a statechart diagram. States are represented as nodes, and actions (with their conditions) are represented as edges. Each action has an initial state and a subsequent next state. If a trigger is present, the edge represents an implication (see Figure 2).

3. Evaluation of RSCharter

3.1. Evaluation Measure: Accuracy

We evaluate RSCharter in terms of accuracy performance. We propose two accuracy measures as follows:

¹We assume an existing RTN module for this translation

Table 1

FLC Rule Table: Rules for converting extracted Trigger and Action Clauses (in NL) into First-Order-Predicate expressions.

FOPL Conversion of Trigger Clause		
FOPL Conversion of Existential Trigger Clause	FOPL Conversion of Comparative Trigger Clause	
CASE 1: is-a	CASE 2: Non is-a	
X is a $y_{POS:JJ}$	$X < \text{action} >_{POS:VB} y_{POS:JJ/ADJ}$	$x < \text{comparative} >_{POS:JJR/RBR} y_{POS:JJ/ADJ}$
$\exists x, y(X(x) \wedge \text{STATUS}(y) \wedge \text{hasStatus}(x, y))$	$\exists x, y(X(x) \wedge \text{STATUS}(y) \wedge \text{action}(x, y))$	$\exists x, y(X(x) \wedge Y_{POS:noun-form}(y) \wedge \text{comparative}(x, y))$
$x_{POS:PRP}$ is a $y_{POS:PRP}$	$x_{POS:PRP} < \text{action} >_{POS:VB} y_{POS:PRP}$	$x_{POS:PRP} < \text{comparative} >_{POS:JJR/RBR} y_{POS:PRP}$
$\forall x(\text{AGENT-1}(x) \rightarrow \text{AGENT-2}(x))$	$\exists x, y(\text{AGENT-1}(x) \wedge \text{AGENT-2}(y) \wedge \text{action}(x, y))$	$\exists x, y(\text{AGENT-1}(x) \wedge \text{AGENT-2}(y) \wedge \text{comparative}(x, y))$
$X_{POS\#JJ/PRP}$ is a $Y_{POS\#JJ/PRP}$	$X_{POS\#JJ/PRP} < \text{action} >_{POS:VB} Y_{POS\#JJ/ADJ/PRP}$	$X_{POS\#JJ/PRP} < \text{comparative} >_{POS:JJR/RBR} Y_{POS\#JJ/ADJ/PRP}$
$\forall x(X(x) \rightarrow Y(x))$	$\exists x, y(X(x) \wedge Y(y) \wedge \text{action}(x, y))$	$\exists x, y(X(x) \wedge Y(y) \wedge \text{comparative}(x, y))$
FOPL Conversion of Action Clause		
$X < \text{token} >_{POS:MD} < \text{action} >_{POS:VB} Y$		
Initial State: $\exists x, y X(x) \wedge Y(y) \wedge \neg \text{action}(x, y)$	Action (Edge): $\text{action}(x, y)$	Next State: $\exists x, y X(x) \wedge Y(y) \wedge < \text{has} \oplus \text{action}_{POS:particle-form} > (x, y)$

$\exists x, y(\text{db module}(x) \wedge \text{data}(y) \wedge \neg \text{extract}(x, y)) \rightarrow \exists x, y(\text{database}(x) \wedge \text{populated}(y) \rightarrow \text{extract}(x, y)) \rightarrow \exists x, y(\text{db module}(x) \wedge \text{data}(y) \wedge \text{has extracted}(x, y))$

Figure 2: Example output of SDG given Trigger FOPL expression (edge label) and Action FOPL expressions (initial and final states; see Table 1.)

Table 2

Evaluation Dataset: Statistics with corresponding FOPL complexity & Accuracy Performance of RSCharter-R1

SRS documents	SRS domain	Conditional Reqs.	Non-Conditional Reqs.	Ground-truth FOPL Complexity		Exact Precision	Exact Recall
				No. of Operators	No. of Operands		
home 1.3	Digital Home	05	20	126 (avg.: 5.04)	166 (avg.: 6.64)	0.84	0.81
hats	Assur. Transformation Sys.	44	70	557 (avg.: 4.88)	708 (avg.: 6.21)	0.91	0.90
esa	Space Data Mgmt.	01	08	37 (avg.: 4.11)	54 (avg.: 6.00)	0.83	0.78
agentmom	Multi-agent Sys.	00	24	106 (avg.: 4.41)	141 (avg.: 5.87)	0.90	0.92
sprat	Goals & Scenario Mgmt.	09	126	590 (avg.: 4.37)	793 (avg.: 5.87)	0.94	0.93
blit draft	Laboratory Info. Sys.	08	26	154 (avg.: 4.52)	179 (avg.: 5.26)	0.92	0.90
Total		67	274	1570 (avg.: 4.6)	2041 (avg.: 5.98)	(avg.:) 0.89	(avg.:) 0.87

Definition 1 (Exact Precision (EP)). *Exact Precision (EP) is the ratio between the number of exact predictions (α) and the total number of predictions by RSCharter (including incorrect ones denoted as β), where exact prediction means an **exact match** between predicted and ground-truth. Hence, $EP = \frac{\alpha}{\alpha + \beta}$.*

It is to be noted that in the above definition, an **exact match** implies a **complete match over the entire syntax** of the predicted and target FOPL expression, including predicates and their corresponding arguments.

Definition 2 (Exact Recall (ER)). *Exact Recall (ER) is the ratio between the number of exact predictions by RSCharter (α) and the total number of ground-truth FOPL expressions (including missed ones denoted as γ). Hence, $ER = \frac{\alpha}{\alpha + \gamma}$.*

3.2. Evaluation Dataset

To evaluate RSCharter accuracy, we use the PuRE dataset (Ferrari et al., 2017), which contains six SRS documents from various domains² (see Table 2). RT compliance normalization is performed with AutoReco by Balwani et al. (2023). Ground-truth FOPL expressions were manually created, categorized as functional vs. non-functional and conditional (60) vs. non-conditional (299)³. Complexity is assessed by the number of logical operators and predicates (see Table 2).

3.3. Baseline Model

As a part of a preliminary case study analysis of the efficacy of the RSCharter framework, we have developed a rule-based baseline model, called RSCharter-R1⁴, along the design principles of RSCharter. We implement the ATE module of RSCharter-R1 using spaCy dependency parser v3.7.1⁵. The FLC module of RSCharter-R1 follows the rule-set defined in Table 1. We use GraphWalker v4.3.2⁶ as a state-chart diagram generator to implement the SDG module of RSCharter-R1.

²Including public companies, product standards, and university projects

³The augmented dataset will be open-sourced upon manuscript acceptance

⁴R-1 stands for Rule-based v1.0

⁵spaCy Dependency Parser v3.7.1

⁶GraphWalker v4.3.2

3.4. Preliminary Observations

We observe that RSCharter-R1 exhibits very high EP (best: 0.94; avg.: 0.89) and ER (best: 0.93; avg.: 0.87) on all the SRS documents except *esa* and, to a certain extent, *Home1.3*. On further analysis, we find that the overall drop in accuracy can be attributed to a lack of accuracy in dependency parsing of more complex statements having more logical operands.

4. Discussions and Future Work

This paper presents RSCharter a framework for generating statechart diagrams from natural language requirement specifications by translating them into First-Order Predicate Logic (FOPL) expressions. The framework is implementation-agnostic, with accuracy dependent on dataset complexity, parser accuracy, and FOPL conversion rules. Future work aims to improve RSCharter with enhanced datasets, advanced augmentation techniques, and a broader metric suite for evaluating statechart diagram generators.

Acknowledgement

This work is supported by the Space Applications Centre (SAC), Indian Space Research Organisation (ISRO) under the 'RESPOND' program.

References

- Ahmed, S., Ahmed, A., Eisty, N.U., 2022. Automatic transformation of natural to unified modeling language: A systematic review, in: 2022 IEEE/ACIS 20th International Conference on Software Engineering Research, Management and Applications (SERA), pp. 112–119.
- Arora, C., Sabetzadeh, M., Briand, L., Zimmer, F., 2016. Extracting domain models from natural-language requirements: approach and industrial evaluation, in: Proceedings of the ACM/IEEE 19th International Conference on Model Driven Engineering Languages and Systems, Association for Computing Machinery, New York, NY, USA. p. 250–260.
- Balwani, S., Tiwari, S., Dasgupta, S., Sharma, A., 2023. Autoreco: A tool for recommending requirements for their non-conformance with requirement templates (rts), in: 2023 IEEE 31st International Requirements Engineering Conference (RE), pp. 365–366.
- Bozyigit, F., Bardakci, T., Khalilipour, A., Challenger, M., Ramackers, G., Babur, Ö., Chaudron, M.R., 2024. Generating domain models from natural language text using nlp: a benchmark dataset and experimental comparison of tools. *Software and Systems Modeling*, 1–19.
- Ferrari, A., Spagnolo, G.O., Gnesi, S., 2017. Pure: A dataset of public requirements documents, in: 2017 IEEE 25th International Requirements Engineering Conference (RE), pp. 502–505.
- de Marneffe, M.C., Dozat, T., Silveira, N., Haverinen, K., Ginter, F., Nivre, J., Manning, C.D., 2014. Universal Stanford dependencies: A cross-linguistic typology, in: Proceedings of the Ninth International Conference on Language Resources and Evaluation (LREC'14), European Language Resources Association (ELRA), Reykjavik, Iceland. pp. 4585–4592.